

NIPT 的时点选择与胎儿异常判定的研究

摘 要

无创产前检测 (NIPT) 在胎儿染色体异常筛查中具有重要意义, 其准确性与胎儿染色体浓度及检测时点选择密切相关。本文基于某地区高 BMI 孕妇 NIPT 数据, 围绕胎儿染色体浓度关联特性、在不同 BMI 分组下的检测时间选择及女胎异常判定展开研究, 研究成果为 NIPT 临床应用提供科学依据, 可降低孕妇潜在风险。

针对问题一, 在分析胎儿 Y 染色体浓度与孕妇孕周数和 BMI 等指标的相关特性前, 首先进行数据预处理, 计算缺失值 BMI, 修正孕期形式, 并对末次月经缺失值进行剔除。然后在数据处理的基础上, 绘制散点图、计算 *Spearman* 相关系数以初步探索 Y 染色体浓度与各变量间的关系, 得到变量间非线性的结论。基于该发现, 分别建立多项式回归模型和非线性混合效应模型表示 Y 染色体浓度与变量的关系。结果显示, 非线性混合效应模型拟合程度更好, $R^2 = 0.7980$, $p \leq 0.001$, $MSE = 0.0023$, 说明该模型预测精确, 显著性强。

针对问题二, 本文需依据 BMI 进行合理分组, 在各组间寻找最优 NIPT 时点。先采用 **K-means 算法**, 对男胎孕妇的 BMI 和男胎 Y 染色体浓度进行聚类得到 BMI 的分布区间, 但鉴于分组过程出现组间重叠现象导致误差较大。进而, 我们设计了一种以组内 Y 染色体浓度的方差最小化为目标的**优化模型**, 通过**动态规划算法**进行求解, 得到更优 BMI 分组区。之后, 建立孕妇的风险量化模型, 以最小化风险为目标求解各组 NIPT 最佳时点。最后, 根据 3σ 原则和**蒙特卡洛模拟**预测 Y 染色体浓度的真实值, 并代入上述模型, 得到更正误差后的 NIPT 最佳时点在 BMI[20.70,31.22]区间为 11.00,[31.22,33.20]区间为 11.00, [33.20,+ ∞]区间为 11.14。

针对问题三, 在问题二的基础上, 同时考虑身高、年龄、体重等因素对胎儿 Y 染色体浓度达标时间的影响, 与问题二采用类似的方法, 建立最小化组内 Y 染色体浓度方差为目标的优化模型, 得到 BMI 的分组区间。随后, 考虑二分类变量 (胎儿 Y 染色体是否达标), 我们根据 **Logistic 回归函数**建立概率模型, 类比问题二的方法, 求得最佳 NIPT 时点并检测误差, 得到更正误差后的 NIPT 最佳时点在 BMI[20.7,30.2]区间为 11.40, [30.2,31.9]区间为 16.63, [31.9,33.9]区间为 18.94,[33.9,+ ∞]区间为 15.00。

针对问题四, 考虑 X 染色体, 21 号、18 号、13 号染色体的 Z 值、GC 含量、读取段数及相关比例、BMI 等因素, 以 21 号、18 号、13 号染色体非整数倍为判定结果, 建立女胎异常的判定方法。根据问题三的启发, 首先考虑 Logistic 回归在本问题中的适用性, 建立 Logistic 回归模型对女胎异常情况进行判定, 并对模型进行评估。但考虑到逻辑回归无法处理交互作用, 且 GC 含量数据存在噪声, 进而尝试建立**随机森林模型**进行再次判断与评估, 发现随机森林模型性能更好 (准确率=0.9125, 召回率=0.9125, F1 分数=0.8706), 因此选用建立的随机森林模型对女胎是否异常进行判定。

关键词: 非线性混合效应模型 蒙特卡洛模拟 Logistic 回归 随机森林

一、问题重述

1.1 问题背景

为了解胎儿健康状况,实现对畸形胎儿的早发现、早治疗,产前检测尤为重要。NIPT检测作为一种重要的产前检测技术,其原理是通过检测母体血液中胎儿游离的 DNA 片段,来判断染色体是否异常。该项技术的准确性主要取决于胎儿性染色体浓度,当男胎 Y 染色体浓度大于等于 4%或女胎 X 染色体浓度正常时,我们认为 NIPT 结果较为可靠。与此同时,为了降低治疗风险,应在保证准确性的前提下尽可能提早进行 NIPT 检测。由于 Y 染色体浓度同时受到孕周数和 BMI 影响,一般需依据 BMI 值对孕妇人群进行分组,以确定不同组别的最佳 NIPT 检测时点。在此背景下,探究如何依据 BMI 对孕妇进行合理分组,确定最佳检测时点,对于提高产前检测的准确性,降低后续治疗风险具有重要意义。

1.2 问题提出

现基于以上背景及附录相关信息我们需要建立数学模型解决以下问题:

- 1) 分析胎儿 Y 染色体浓度与孕妇孕周数和 BMI 等指标的相关特性,建立关系模型并检验其显著性。
- 2) 以最小化孕妇的潜在风险为目标,将男胎孕妇根据 BMI 进行合理分组,确定每组的 BMI 区间及最佳 NIPT 时间点,并分析检测误差对结果的影响。
- 3) 综合考虑身高、体重、年龄等因素对 Y 染色体浓度达标时间的影响,及检测误差和达标比例,将男胎孕妇根据 BMI 重新进行合理分组,确定新的每组的 BMI 区间及此时最佳 NIPT 时间点,使风险最小化,并分析检测误差对结果的影响。
- 4) 考虑到 Y 染色体无法作为女胎是否异常的判定标准,要求基于女胎孕妇 21 号、18 号和 13 号染色体非整倍体结果及其他一些相关因素(如 X 染色体和上述染色体的 Z 值、GC 含量等),设计出一种判定女胎异常的方法。

二、问题分析

2.1 问题一的分析

针对问题一,我们按照如下步骤进行:建立胎儿 Y 染色体浓度与孕妇孕周数和 BMI 等指标的关系模型,并对模型进行显著性检验。

首先,对数据进行预处理,包括计算缺失值 BMI,剔除末次月经空白值等操作。在此基础上通过绘制散点图能够大致观察到数据呈现非线性的特点,进而我们采用斯皮尔曼相关系数初步探索 Y 染色体浓度与各变量间的关系,再次分析绘图结果,我们可以进一步验证变量间关系是非线性的结论。因而,基于这一发现分别建立多项式回归模型和非线性混合效应模型对变量关系进行拟合,且采用 5 折交叉验证的方法评估模型的稳固性,最后通过 F 检验和 t 检验进行显著性分析。

2.2 问题二的分析

针对问题二,题目要求对男胎孕妇的 BMI 进行合理分组,且提及男胎孕妇的 BMI 值与 Y 染色体浓度的最早达标时间存在密切联系。于是本文首先采用 K-means 聚类算法,以男胎孕妇的 BMI 和 Y 染色体浓度这两个变量为指标进行聚类,以确定最佳分组

数。但由于 K-means 聚类得到的 BMI 分布区间存在重叠表现，故此处选择建立一个以总组内 Y 染色体浓度方差最小化为目标的优化模型来确定分组。而后，建立孕妇的风险量化模型，以最小化风险为目标，求解每组的 NIPT 最佳检测时点。查阅资料得知 Y 染色体浓度随机误差的标准差为 0.42%^[1]。结合上述，根据 3σ 原则和蒙特卡洛模拟预测出每个男胎孕妇 Y 染色体浓度的真实值，并利用此真实值进行计算，得到最佳的 NIPT 检测时点和最小风险。将其与原始数据得到的结果进行比较，分析检测误差对结果的影响。

2.3 问题三的分析

针对问题三，要综合考虑身高、体重、年龄等多因素对 Y 染色体浓度的影响，本文在问题二的基础上利用逻辑回归建立概率模型，并将其代入以最小化组内 Y 染色体浓度方差为目标的目标函数中求解最佳分组区间。然后，类比问题二的方法，建立风险量化模型，以最小化风险为目标求得最佳 NIPT 时点。并利用蒙特卡洛模拟检测误差，得到更正误差后的 MBI 分组区间及其 NIPT 最佳时点。

2.4 问题四的分析

针对问题四，要找到一种检测女胎异常的判定方法，女胎相对于男胎没有 Y 染色体，传统模型失效。首先考虑问题二、三中对于二分类目标变量的 Logistic 回归模型可否沿用或改进，Logistic 回归仅通过系数绝对值判断重要性，对于交互作用无法处理，更无法对交互贡献量化，且对 GC 异常等噪声值敏感，容易误判。因此我们选择一种能够更好处理特征关系，具有强噪声耐受性，易解释特征重要性的随机森林机器学习方法。随机森林通过 Bootstrap 采样和随机特征子集降低方差，由多棵决策树集成式学习，捕捉时、空复杂交互作用，非线性拟合能力强，可有效解决上述问题，精准满足临床判定需求。GC 值分组、特征标准化等数据预处理后，建立特征工程，构建随机森林模型预测女胎染色体非整倍体类型，而后通过精确率、召回率、加权 F_1 分数等指标评价模型。最后通过机器学习得到女胎孕妇的 21 号、18 号和 13 号染色体非整数倍是否异常条数的预测概率，作为女胎异常判断依据。

三、模型假设

1. 假设时点 t 的风险系数与孕期呈正相关；
2. 假设孕妇间相对独立，孕妇间的测量数据互不干扰；
3. 假设胎儿的游离 DNA 片段在一段时间内含量相对稳定；
4. 假设附件中提供的孕妇年龄、身高、体重、BMI 等数据是准确无误的；
5. 假设所用数据能代表研究区域内孕妇的人群特征，进而模型基于该样本的结论可推广到目标人群。

四、符号说明

符号	说明	单位
R	风险系数	/
z_i	第 i 个样本标准化后的值	/
y	Y 染色体浓度	/
X_{BMI}	第 i 个样本的 BMI 值	/
S	平均轮廓系数	/
N	样本数	个
t	孕周	周

五、模型的建立与求解

5.1 问题一模型的建立与求解

5.1.1 数据预处理和变量关系的初步探索

在分析胎儿 Y 染色体浓度与孕妇的孕周数和 BMI 等指标的相互关系前，必须先进行数据的预处理，这一步骤对于确保分析结果的准确性与可靠性至关重要。该过程主要包括三部分，一是将附件中男胎孕妇的原始孕周数据从“周数+天数”的复合单位格式转化为以周为单位的连续小数形式（例如将 11w+6 化为 11.86），二是：将末次月经为空白值的 12 项数据剔除，三是：将染色体的非整倍体空白处标记为正常。

在完成数据的预处理后，为了对变量之间的关系初步定性，为选择合适的模型提供指导方向，本文首先对 Y 染色体浓度与多项指标的关系进行初步探索并绘制了散点图进行可视化分析。

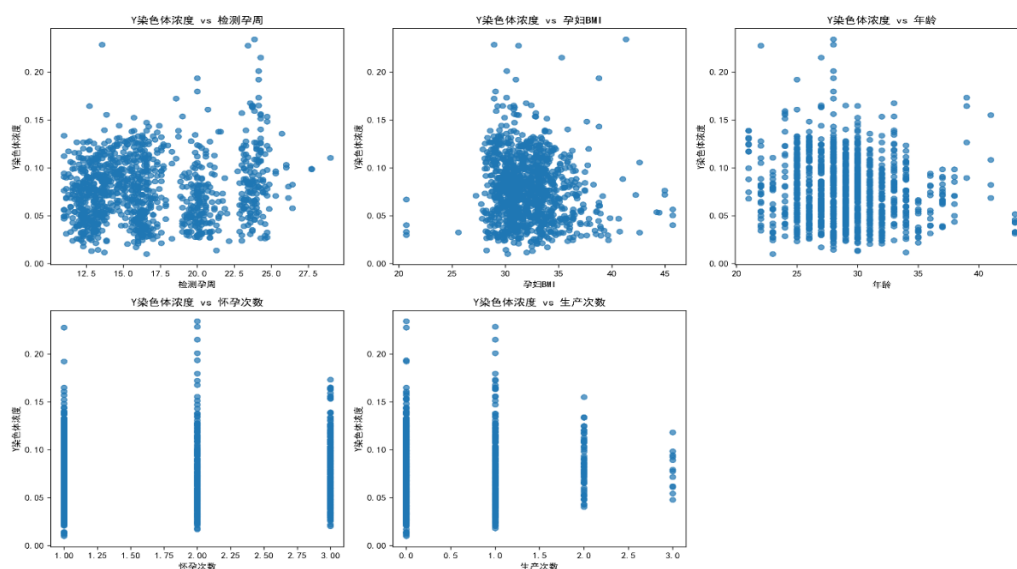


图 1 Y 染色体浓度与 5 个因素的散点图

通过对图 1 中散点图的观察可以发现：散点在整个坐标区域内分布广泛，且总体呈现明显的纵向分布特点。这一分布形态反应了不同个体的 Y 染色体浓度间存在显著差异且变量间显然不满足线性关系，无法用直线进行拟合。因此，需要寻找合适的回归模型以反映变量的非线性关系和个体显著性差异。

5.1.2 关系模型的建立

首先问题一的题干明确 Y 染色体浓度与孕妇的孕周数和 BMI 指标具有相关特性，再结合文献资料和已知生物学知识，预测孕妇的年龄、怀孕次数、生产次数可能会对胎儿的 Y 染色体浓度产生影响，据此选择该五项指标进行如下相关性分析，对强相关性的指标作进一步的回归分析。

1.Spearman 相关系数

为描述变量之间的非线性关系，本文首先计算了 Y 染色体浓度与筛选出的多项指标的 Spearman 相关系数，并依据相关系数矩阵绘制热力图，直观呈现多变量间的线性关联强度。

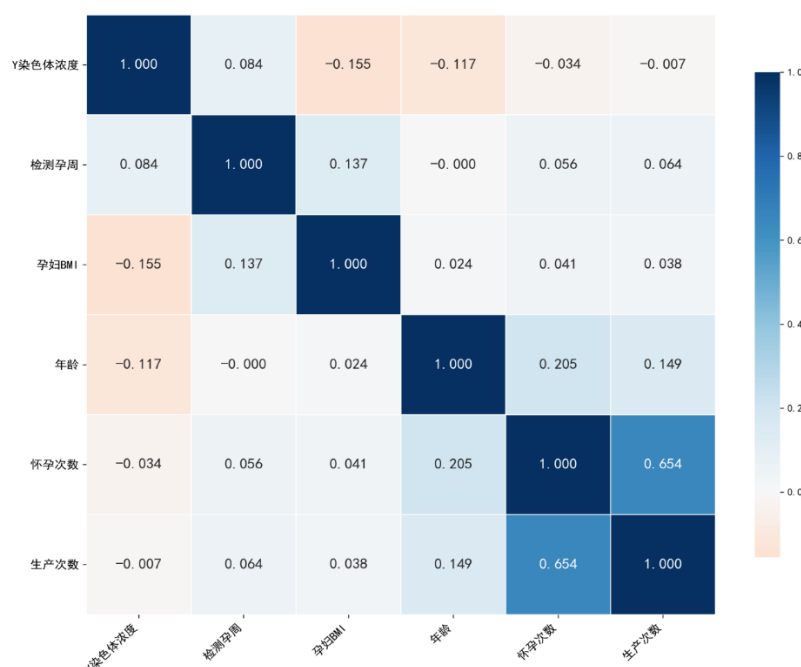


图 2 相关矩阵热力图

通过观察图 2 发现 Y 染色体浓度与各指标相关系数的绝对值最大不足 0.2，说明变量间的线性相关性较弱。在此基础上，又计算了斯皮尔曼相关系数的 p 值，得到 Y 染色体浓度与检测孕周、孕妇 BMI、年龄的 p 值均小于 0.05，说明 Y 染色体浓度与这些变量的单调相关性极显著。而与怀孕次数、生产次数的 p 值均大于 0.05，说明它们的单调相关性不显著。而后的回归分析将无需考虑这些指标。

2.多项式回归

考虑到多项式回归模型对非线性数据的拟合效果较好，本研究又进一步选用多项式回归模型进行拟合。选取了上述结果中与 Y 染色体浓度相关性显著的三个指标孕周、孕妇 BMI、年龄作为自变量，依次建立了一次、二次、三次多项式回归模型。建立的多项式回归模型通式为：

$$\hat{y} = \beta_0 + \sum_{i=1}^p \beta_i x_i + \sum_{1 \leq i < j \leq p} \beta_{ij} x_i x_j + \sum_{1 \leq i < j < k \leq p} \beta_{ijk} x_i x_j x_k + \dots + \beta_{12\dots p} x_1 x_2 \dots x_p + \varepsilon \quad (1)$$

其中：\$\hat{y}\$ 表示因变量 Y 染色体浓度的预测值；\$x_1, x_2, \dots, x_p\$ 为表示研究的不同自变量，在本题中 \$p\$ 的值取 3，自变量分别为检测孕周、孕妇 BMI 和年龄；\$\beta_0, \beta_i, \beta_{ij}, \dots\$ (\$\beta\$ 系列) 表示待预测的回归系数；\$\varepsilon\$ 表示随机误差项。

通过计算发现决定系数 (\$R^2\$) 随模型复杂度的增加而依次上升，其中决定系数最高的 3 次多项回归的拟合曲线如图所示，决定系数为 0.1108。解得的只保留系数绝对值大于 0.001 项的三次回归方程为：

$$y_c = -2.2305 + 0.1439x_{\text{BMI}} + 0.1161x_w + 0.0066x_{\text{age}} - 0.0036x_{\text{BMI}}^2 - 0.0354x_w^2 - 0.0025x_{\text{BMI}}x_w \quad (2)$$

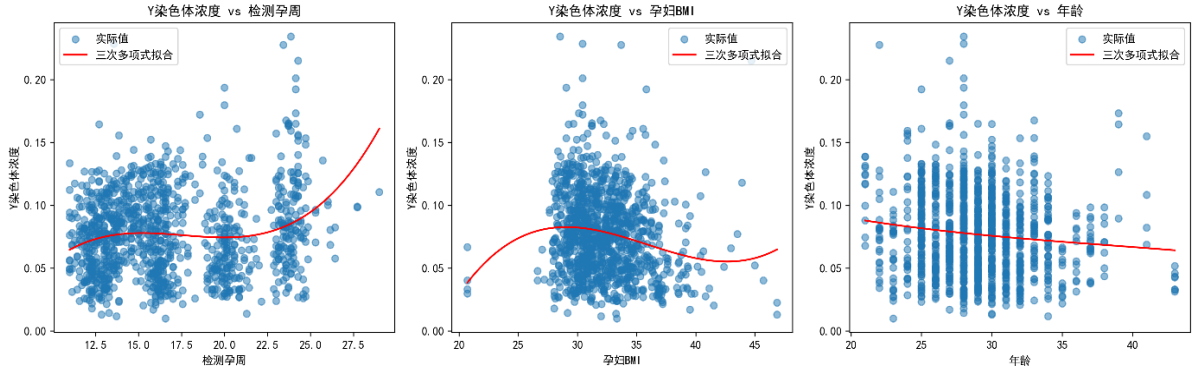


图 3 Y 染色体浓度与各变量关系的三次多项式拟合

图 3 中的拟合曲线及其得到的相关系数显示 Y 染色体浓度与选取的指标相关性较弱，无法很好地对数据进行拟合。

3.非线性混合效应模型

对于一些结构较为复杂的数据类型，例如本文中给到的不同孕妇在不同孕周的检测数据，其属于重复测量数据，它呈现出很强的层次性结构，并且由于对同一位孕妇进行了多次的测量，这会使数据在个体间存在一定的内在相关性。对于这类具有重复性、且呈现非线性的数据，为了更好地描述数据间的关系，又尝试采用混合效应模型进行建模。

非线性混合效应模型会同时考虑固定效应与随机效应，在本题中，将检测孕周、孕妇 BMI 和年龄作为固定效应，孕妇编号作为随机效应。

该模型的表达式为：

$$y_{c_i,j} = \beta_0 + \beta_1 \cdot x_{w_i,j} + \beta_2 \cdot x_{\text{BMI}_i,j} + \beta_3 \cdot x_{\text{age}_i,j} + u_i + \varepsilon_{i,j} \quad (3)$$

其中：\$i\$ 表示孕妇编号 (\$i = A001, A002, \dots\$)；\$j\$ 表示第 \$i\$ 个孕妇的第 \$j\$ 次测量；\$\beta_0\$ 表示固定效应截距；\$\beta_1, \beta_2, \beta_3\$ 表示固定效应斜率；\$x_{w_i,j}, x_{\text{BMI}_i,j}, x_{\text{age}_i,j}\$ 为第 \$i\$ 个孕妇第 \$j\$ 次测量的检测孕周数、BMI 值和年龄；\$u_i\$ 随机效应（随机截距），代表第 \$i\$ 个孕妇的独特效应，且 \$u_i \sim N(0, \sigma_u^2)\$；\$\varepsilon_{i,j}\$ 表示第 \$i\$ 个孕妇第 \$j\$ 次测量的随机误差，且 \$\varepsilon_{i,j} \sim N(0, \sigma_\varepsilon^2)\$。

通过拟合得到固定效应截距 \$\beta_0\$ 为 0.096922，检测孕周的固定效应斜率 \$\beta_1\$ 为 0.003125，孕妇 BMI 的固定效应斜率 \$\beta_2\$ 为 -0.001383，孕妇年龄的固定效应斜率 \$\beta_3\$ 为 -0.000934。得到的 \$y_{c_i,j} = 0.096922 + 0.003125 \cdot x_{w_i,j} - 0.001383 \cdot x_{\text{BMI}_i,j} - 0.000934 \cdot x_{\text{age}_i,j} + u_i + \varepsilon_{i,j}\$，混

合效应模型的预测值与实际值的对比图如图 4 所示：

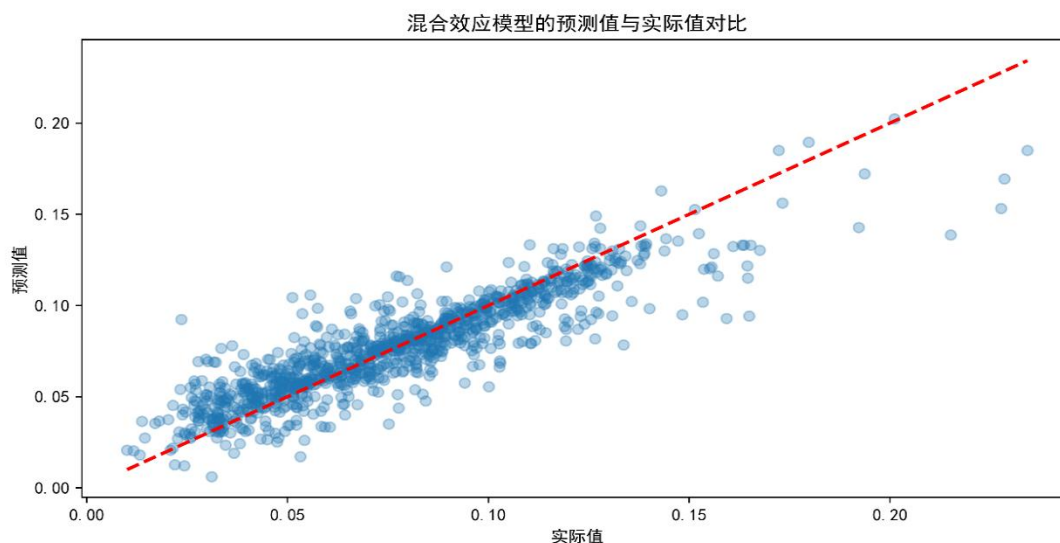


图 4 混合效应模型的预测值与实际值对比

观察图 4 发现预测值与实际值近似落于直线两侧或直线上，说明该混合效应模型对数据的拟合效果较好。

5.1.3 关系模型的显著性检验

1. 对多项式回归模型的检验

1) 拟合度检验

模型的拟合度 R^2 为 0.115

2) 方差分析检验：

通过 F 检验来评估模型的显著性，检验自变量整体对因变量的影响程度。

本题利用 SPSS 得到 $F = 7.269$ ， $p = 9.98 \times 10^{-19}$ ，其显著性远小于 0.01，表明所构建的三元多项式回归关系模型整体上是极其显著的。

3) 偏回归系数显著性检验：

通过 t 检验来检测每个偏回归系数是否不为 0，若显著不为 0，说明该偏回归系数所在项（包括一次项、多项式项和交互项）对因变量有显著影响。

表 1 偏回归系数的显著性检验结果

偏回归系数所在项	t 值	p 值
X_1	2.5508	1.0887×10^{-2}
X_2	2.2497	2.4670×10^{-2}
X_4	-3.2076	1.3784×10^{-3}
X_7	-3.2625	1.1396×10^{-3}
X_{10}	4.7707	2.0924×10^{-6}
X_{13}	3.6060	3.2546×10^{-6}
X_{16}	3.6728	2.5190×10^{-4}

系数显著性检验可知 $X_1, X_2, X_4, X_7, X_{10}, X_{13}, X_{16}$ 的偏回归系数显著，如表 1 所示 p 值均小于 0.05，而其余项则并不显著。

2. 对非线性混合效应模型的检验

1) 随机效应显著性检验(似然比检验)

检测统计量 $\chi^2=674.19$, $p<0.001$, 这个极其显著的 p 值意味着数据的随机效应巨大, 不容忽视, 采用该非线性混合效应模型显著优于普通线性回归模型。且检测得到 $MSE=0.00023$ 的值很小, 表明模型的预测很精准, $R^2=0.7980$ 反应了该模型的解释力很强, 能解释大部分变异。

2) 固定效应显著性检验

通过检验发现检测孕周 p 值为 4.92×10^{-84} , BMI 的 p 值为 0.0075, 均小于 0.01, 说明孕和 BMI 为显著影响因素。而年龄的 p 值为 0.0634, 说明年龄影响并不显著。

5.2 问题二模型的建立与求解

结合分析, 解决问题二将如下进行: 一是对男胎孕妇的 BMI 进行合理分组, 二是计算每组使孕妇潜在风险最低的最佳 NIPT 时点, 三是分析检测误差对结果的影响。

5.2.1 对 BMI 的合理分组

1. 结合 K-means 聚类 and 轮廓系数分组

为将男胎孕妇的 BMI 划分为相似的簇, 本研究试选择 K-means 算法, 其作为一种聚类算法, 核心思想是根据给定的聚类数 k , 把 n 个对象划分为 k 簇, 使每个簇内的样本尽可能相似 (即簇内方差最小), 从而实现合理分类的目的。进一步, 可以从 Y 染色体浓度与 BMI 的散点图图 5 观察到其形状近似为圆形, 是球形簇, 适合用轮廓系数对聚类结果进行评估以确定最佳聚类数 k 值。以下是该方法的步骤:

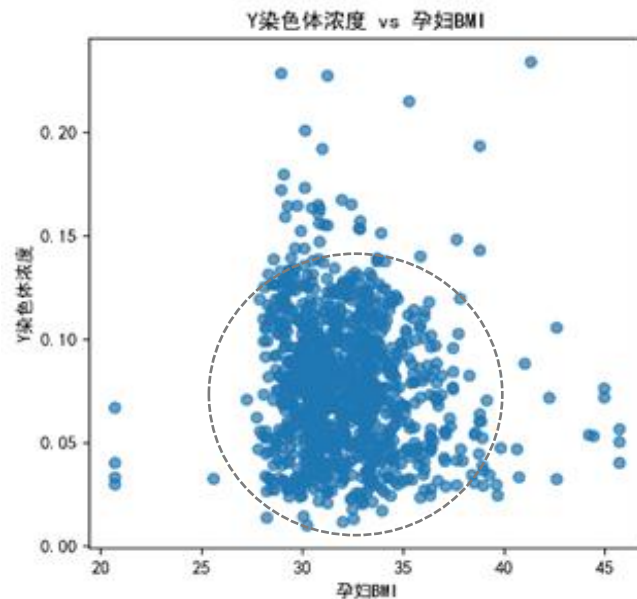


图 5 Y 染色体浓度与 BMI 的散点图

Step1.数据预处理

对数据 BMI 和 Y 染色体浓度进行标准化处理, 以消除量纲的影响。

$$x'_i = \frac{x_i - \bar{x}}{s} \quad (4)$$

其中 x_i 为原始数据, $\bar{x}$ 为该数据的均值, s 为标准差。

Step2.设定聚类数的范围

确定待测的 k 值范围为2~5，由于轮廓系数对单簇无意义，所以避免 $k=1$ 。

Step3.初始化

随机选取 k 个样本点作为聚类中心。

Step4.计算欧氏距离

计算每个样本到这 k 个中心的欧氏距离，根据距离最近原理将样本分至这 k 个类别中距离最近的簇中。

$$d = \sqrt{\left(x_{\text{BMI},i}' - c_{\text{BMI},k}\right)^2 + \left(x_{c,i}' - c_{c,k}\right)^2} \quad (5)$$

其中 $(x_{\text{BMI},i}', x_{c,i}')$ 为样本的标准化特征， $(c_{\text{BMI},k}, c_{c,k})$ 为第 k 簇的标准化质心。

Step5.迭代更新聚类中心：

重新计算每个簇中数据的平均值点，将其作为新的聚类中心。

Step6.重复步骤4和5：

重复步骤4和5直至质心的位置不再发生显著变化。

Step7.轮廓系数：

首先是计算当前取值 k 下每个样本的轮廓系数 s_i 。

$$s_i = \frac{b_i - a_i}{\max(a_i, b_i)} \quad (6)$$

其中 $s_i \in [-1, 1]$ ， s_i 越接近1代表样本的聚类效果越好，越接近0则代表样本处于两簇边界。负值代表样本被分到了错误的簇中。

然后是计算该 k 值下的平均轮廓系数 S ：

$$S = \frac{1}{n} \sum_{i=1}^n s_i \quad (7)$$

Step8.确定最优 k 值：

遍历 k 的所有不同取值（2~5），重复步骤3到步骤7，得到每个 k 值对应的平均轮廓系数 S 如表2所示，并绘制折线图进行了可视化，选取其中 S 取最大值时对应的 k 值作为BMI分组的最佳聚类数。

表2 聚类数为2~5的平均轮廓系数汇总

聚类数	2	3	4	5
轮廓系数	0.566	0.561	0.572	0.533

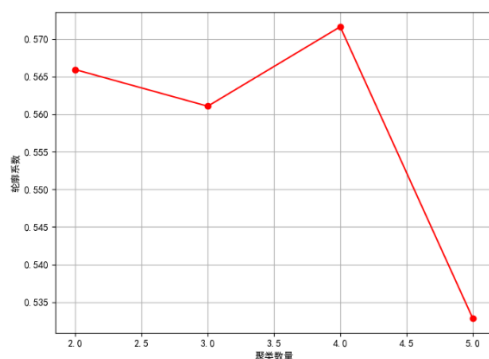


图6 轮廓系数变化折线图

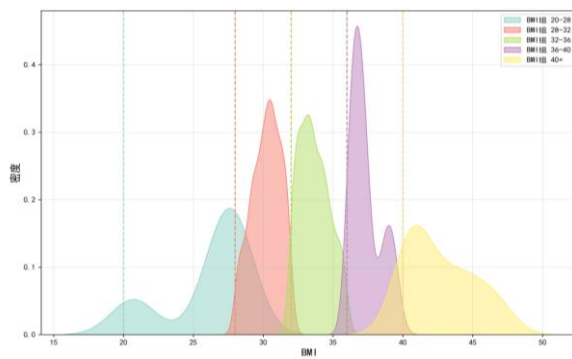


图7 各组BMI分布图

观察经过上述步骤得到的轮廓系数随聚类数量的变化折线图 6 可知：该问 BMI 的最佳聚类数为 4，即应对 BMI 分 4 组。但对得到的聚类结果进行分析发现，BMI 分组结果出现组间重叠如图 7，此处所得结果仅通过恰取两线交点作为临界，此法可解释性差，主观性强，误差较大，效果明显不佳。

因此，下面我们尝试探索其他合理的量化分组方法，考虑同样使组内数据方差尽可能小，且能规避上述组间重叠问题。

2. 基于优化算法的分组

由于聚类效果不佳，本研究又采取了优化算法进行分组，类似地以最小化总组内方差为目标函数，考虑组内达标率尽可能高作为限制条件，通过动态规划算法求解。

步骤如下：

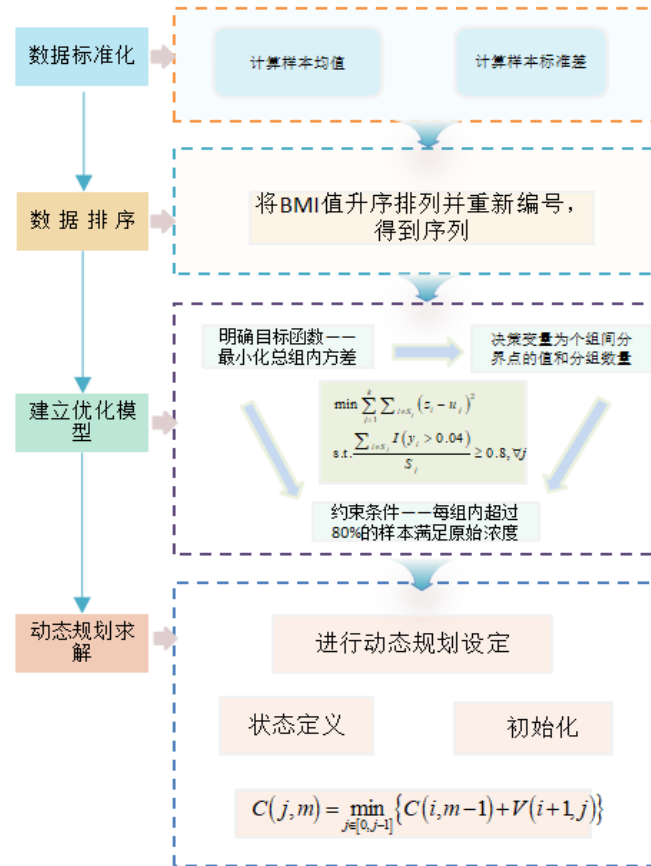


图 8 优化算法流程图

Step1.数据标准化

首先，计算样本均值 u 和样本标准差 σ ：

$$u = \frac{1}{n} \sum_{i=1}^n y_{c,i}$$

$$\sigma = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (y_i - u)^2} \quad (8)$$

然后对每个样本的 Y 染色体浓度进行标准化处理：

$$z_i = \frac{y_i - u}{\sigma} \quad (9)$$

其中 y_i 表示第 i 个样本的 Y 染色体浓度， z_i 表示第 i 个样本标准化后的值。

Step2.数据排序

将 BMI 值 x_i 升序排列并重新编号，得到序列： $x_1 \leq x_2 \leq \dots \leq x_n$

Step3.建立优化模型

明确目标函数为最小化总组内方差，决策变量为 $k-1$ 个组间分界点的 BMI 值 t_j 和分组数量 k 。建立目标函数为：

$$\min \sum_{j=1}^k \sum_{i \in N_j} (z_i - u_j)^2 \quad (10)$$

其中 S_j 为第 j 组所包含的样本数量； u_j 为第 j 组组内样本的均值 $u_j = \frac{1}{N_j} \sum_{i=1}^{N_j} z_i$

约束条件为：

$$\frac{\sum_{i \in N_j} I(y_i > 0.04)}{N_j} \geq 0.8, \forall j \in N \quad (11)$$

表示每组内超过 80% 的样本满足原始浓度 $y_i \geq 0.04$ 。

完整的优化模型为：

$$\left\{ \begin{array}{l} \min \sum_{j=1}^k \sum_{i \in N_j} (z_i - u_j)^2 \\ \text{s.t.} \frac{\sum_{i \in N_j} I(y_i > 0.04)}{N_j} \geq 0.8, \forall j \in N \\ I(y_i > 0.04) = \begin{cases} 1, y_i > 0.04 \\ 0, y_i \leq 0.04 \end{cases} \end{array} \right. \quad (12)$$

在此基础上，采用动态规划算法进行求解

求解步骤为：

首先，计算子区间 $[i, j] (1 \leq i \leq j \leq n)$ 内标准化后的 Y 染色体浓度的方差

$$V(i, j) = \frac{1}{j-i+1} \sum_{k=i}^j (z_k - u_{i,j})^2 \quad (13)$$

其中 $u_{i,j} = \frac{1}{j-i+1} \sum_{k=i}^j z_k$ 为区间 $[i, j]$ 的均值

其次，计算合格比例，即 Y 染色体浓度超过 0.04 的样本数占总体样本量的比例

$$P(i, j) = \frac{\sum_{k=i}^j I(y_k > 0.04)}{j-i+1} \quad (14)$$

然后，进行动态规划设定：

状态定义： $C(j, k)$ 表示将前 j 个 BMI 值分为 k 组时，在每组满足浓度约束条件下

能达到的最小总组内方差。

初始化： $C(0,0)=0$ ， $C(j,0)=\infty, \forall j > 0$

在满足各分区 $P(i+1, j) \geq 0.8$ 的条件下建立状态转移方程：

$$C(j, m) = \min_{j \in [0, j-1]} \{C(i, m-1) + V(i+1, j)\} \quad (15)$$

遍历 m 从 1 到 n ，并计算每个 m 对应的 $C(n, m)$ 值。然后，我们选择使 $C(n, m)$ 最小的那个 m 值 3 作为最优的组数，并找到其对应的区间划分，如图 9 所示。区间分别为区间一：20.7~31.22，区间二：31.22~33.20，区间三：33.20 以上。

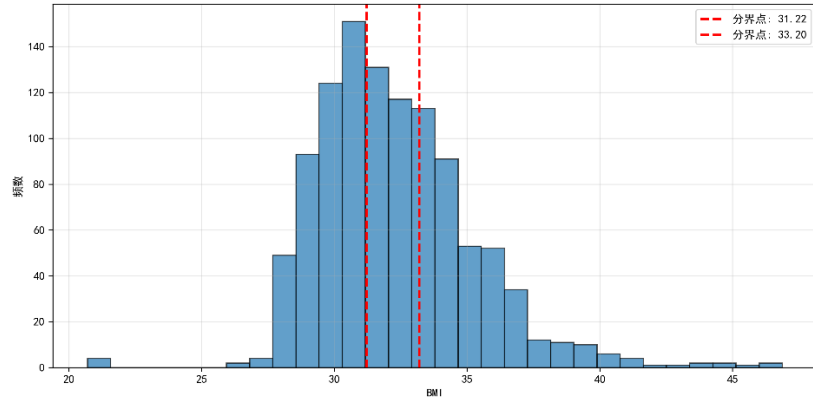


图 9 BMI 与最优分界点分布柱状图

5.2.2 确定最佳 NIPT 时点

在分组后要求确定每组中最佳的 NIPT 时点，达到使孕妇潜在风险最低的目标。考虑到医疗干预早发现、早治疗的特性，可将时间点 t 的单次检测风险量化为两部分：一是在时点 t 检测达标的情况下所承担的 t 时点的固有风险，二是在时点 t 检测未达标时，所承担的因检测延迟到下一时点所造成的下一时点的风险。

根据以上原理，建立时点 t 单次检测风险的量化模型：

$$E(R(t)|t, BMI_g) = R(t) + [1 - P(t, BMI_g)] \cdot R(t + \Delta t) \quad (16)$$

但因为在此处 P 的取值较大，则 $1-P$ 的值相对较小，检测未达标的风险就较小，随着 P 值的增大，检测未达标的风险不断变小，直到可以忽略。而这样就导致，由达标风险决定性影响单次检测风险，甚至仅由达标风险来决定单词检测风险最小，如此预期结果只希望孕周数越小越好，这显然不符合客观实际，无法达到预期效果。因此上述模型存在局限性，得到的最佳 NIPT 时点也不能作为实际临床的检测时点。

所以我们对上述单次检测风险的量化模型作修正，得到如下考虑多次检测风险的量化模型：

$$\left\{ \begin{array}{l} E(R(t)|t, BMI_g) = R(t) + [1 - P(t, BMI_g)] \cdot E(R(t + \Delta t)|t + \Delta t, BMI_g) \\ \quad \quad \quad s.t. \ t + \Delta t \leq 25 \\ \quad \quad \quad t \geq 10 \end{array} \right. \quad (17)$$

其中 t ：监测时点，

BMI_g ：第 g 组 BMI_g 区间

$R(t)$ ：时点 t 的风险系数

$P(t, BMI_g)$: 给定 t 和 BMI_g 时, Y 染色体浓度 $\geq 4\%$ 的概率

由于检测抽血次数最多为 5 次, 所以风险的迭代次数最多为 4 次。

通过在每个 BMI 区间内遍历 10 至 25 周, 搜索使风险最小化的孕周检测时间, 得到如表 3 所示的结果:

表 3 BMI 区间最佳 NIPT 时点统计

BMI 区间	20.70~31.22	31.22~33.20	33.20 以上
最佳 NIPT 时点 (周)	11	11	11.14

5.2.3 分析检测误差

首先, 查阅相关文献资料可知, 用 NIPT 方法检测 Y 染色体浓度时随机误差的标准差为 0.42%。接着, 依据 3σ 原则确定随机误差有 99.73% 的概率 Y 染色体浓度的测量值会落在真实值的 $\pm 1.26\%$ 区间内。而我们知道, 真实值是由测量值和误差决定, 所以反过来, 通过以上关系根据蒙特卡洛原理推断出真实值。具体步骤如下:

Step1: 生成随机样本

基于均值为 0, 标准差为 0.42%, 生成 1 万个可能的随机误差值, 使其根据 3σ 原则, 落在 $\pm 1.26\%$ 的范围内

Step2: 反推真实值

根据真实值=测量值+误差, 可以求得每一个随机样本的真实值。进一步, 对真实值中的极端情况作出取舍, 再对剩下的可信真实值求均值, 即得到预估的 Y 染色体浓度的真实值。

在得到每个样本 Y 染色体浓度对应的真实值后, 计算每个 BMI 区间 Y 染色体浓度大于等于 4% 的概率, 并再次代入风险量化模型求解得到较为准确的 NIPT 最佳检测时点及最小风险期望, 如表 4 所示。绘制了 Y 染色体浓度 $\geq 4\%$ 的比例、最优孕周及最小风险期望这三原始数据与模拟值的柱形图, 如图 9 所示。

表 4 修正误差后问题二的最佳 NIPT 时点及风险期望

BMI 区间	20.70~31.22	31.22~33.20	33.20 以上
最佳 NIPT 时点 (周)	11.00	11.00	11.14
平均最小风险期望	11.1204 $\pm$ 0.0021	11.0891 $\pm$ 0.0017	11.3265 $\pm$ 0.0015

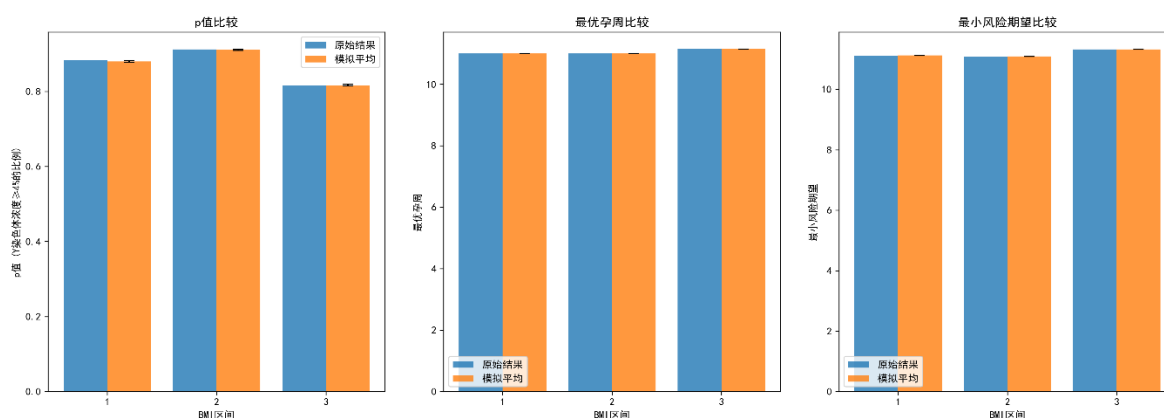


图 10 原始数据与模拟值的对比图

观察图 10, 并对表 3 和表 4, 发现测量误差造成的影响非常微小, 可以忽略不计。其中表 4 的平均最小风险期望表示, 根据蒙特卡洛模拟生成的 100 个样本值分别计算其

最小风险，再求 100 组样本数据的数学期望，得到的此处平均最小风险期望可以较好的反应最佳 NIPT 时点是同样具有较低风险。

5.3 问题三模型的建立与求解

5.3.1 基于优化算法的分组

问题三要求根据身高、体重、年龄、BMI 相关特征及检测误差等对 BMI 进行合理划分。本文采取与问题二类似的优化算法，明确目标函数为最小化总组内方差，决策变量为 $k-1$ 个组间分界点的 BMI 值 t_j 和分组数量 k 。然后对 4 种特征进行标准化。进一步，对所求得的每一种特征的方差进行求和。以最小化 4 种特征的总组内方差为目标函数，每组内超过 80% 的样本满足原始浓度 $y_i \geq 0.04$ 为约束条件，建立了以下优化模型：

$$\begin{aligned} \min & \sum_{j=1}^k \sum_{i \in S_j} \sum_{l=1}^4 (z_{i,l} - u_{j,l})^2 \\ \text{s.t.} & \frac{\sum_{i \in S_j} I(y_i > 0.04)}{S_j} \geq 0.8, \forall j \end{aligned} \quad (18)$$

其中 $z_{i,l}$ 表示第 i 个样本，第 l 种特征标准化后得到的值， $u_{j,l}$ 表示第 j 组第 l 种特征的组内样本的均值 $u_j = \frac{1}{S_j} \sum_{i=1}^{S_j} z_{i,l}$ 。

根据动态规划算法得到使得 4 种特征的总组内方差最小的 BMI 分组结果为：分组数为 4 组，区间划分为区间一：20.7~30.2，区间二：30.2~31.9，区间三：31.9~33.9，区间四：大于 33.9。

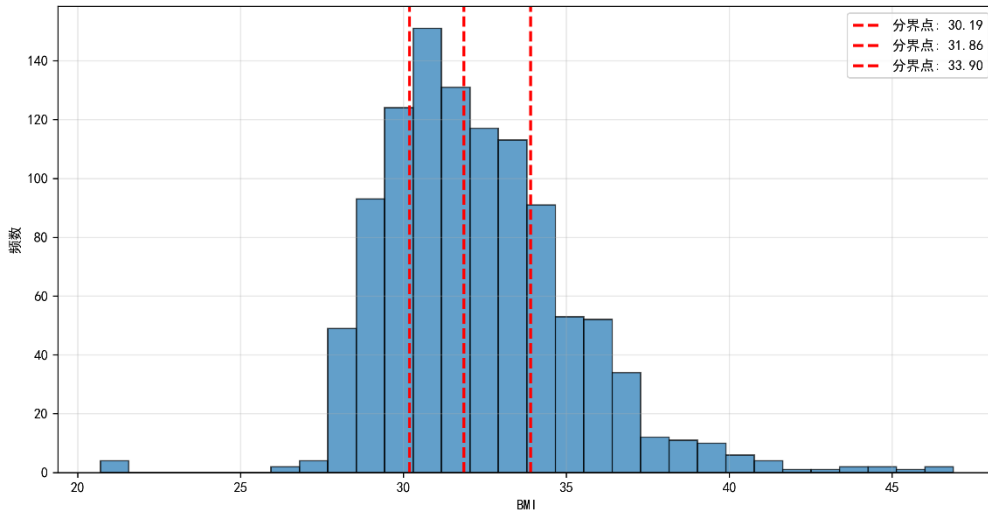


图 11 BMI 频数与最优分界点的分布柱形图

结合 BMI 频数与最优分界点的分布柱形图 11 发现，划分的每组数据量都较为均匀，划分区间比较合理。

5.3.2 确立最佳 NIPT 时点

要确定 NIPT 最佳时点，使孕妇潜在风险最低，建立风险量化模型：

$$E(R(t)|t, X) = R(t) + (1 - P(t, X)) \cdot E(R(t + \Delta t)|t + \Delta t, X) \quad (19)$$

其中 X 为 X_{BMI} 、 X_{age} 、 X_{Height} 、 X_{Weight}

考虑到身高、体重、年龄、BMI 对胎儿的 Y 染色体浓度都有影响，所以本问建立了真实达标概率模型，将前述 4 个特征与 Y 染色体浓度达标的概率建立关系。

首先，将 4 个特征量化为一个总分 β ，用来评判其对 Y 染色体浓度的影响。

$$\beta = \beta_0 + \beta_1 t + \beta_2 X_{\text{BMI}} + \beta_3 X_{\text{age}} + \beta_4 X_{\text{Height}} + \beta_5 X_{\text{Weight}} \quad (20)$$

其次，根据逻辑回归函数建立真实达标概率模型：

$$P(Y=1|t, x) = \frac{1}{1 + \exp(-\beta)} \quad (21)$$

用最大似然估计的方法求解 $\beta_0, \beta_1, \dots, \beta_5$ 的值，得到较为准确的达标概率，代入风险量化模型。并在每个 BMI 区间内遍历 10 至 25 周，搜索使风险最小化的 NIPT 孕周检测时点，得到如表 5 所示的结果：

表 5 问题三 BMI 区间最佳 NIPT 时点统计

BMI 区间	20.7~30.2	30.2~31.9	31.9~33.9	大于 33.9
最佳 NIPT 时点（周）	11	15	17	15

5.3.3 分析检测误差

沿用第二问的蒙特卡洛模拟，对真实的 Y 染色体浓度进行预测并再次代入风险量化模型求解得到较为真实的 NIPT 最佳检测时点及最小期望。

表 6 修正误差后问题三的最佳 NIPT 时点及风险结果

BMI 区间	20.7~30.2	30.2~31.9	31.9~33.9	大于 33.9
最佳 NIPT 时点（周）	11.40±1.20	16.63±1.73	18.94±2.23	15.00±0.44
最小风险期望	27.5	32.2	35.1	54.3

结果表明检测误差会对最佳时点的选择造成小幅度的影响。具体体现在第一个 BMI 区间最佳 NIPT 时点延后了 0.4 周，后面依次延后 1.6 周，延后 1.9 周，第四个 BMI 区间几乎无影响。基本符合 BMI 越大，最佳 NIPT 时点越靠后，且风险随之增大的客观医学规律，可以较好的说明误差检测后模型得到了优化。

5.4 问题四模型的建立与求解

5.4.1 数据预处理与特征工程

1. 对 AB 列异常标签作校准

问题一数据预处理时，已经将 AB 列染色体的非整数倍空白值进行填 0 操作，现继续处理该列数据，将表格中只有一条染色体的数据（例如 T21、T18、T13）标记为 1，将含有两条染色体的数据（例如 T21T18、T18T13）标记为 2，遍阅附件数据未发现附件中 AB 列含有三条染色体的数据， i 表示该样本中有 i 条异常染色体， i 可取 0, 1, 2。

2. 定向处理 GC 异常情况

附录 1 的表格中说明了正常 GC 含量在 40%~60%，而附件中有较多数据不在这个范围中，且主要分布在 0.39 附近，因此我们为了尽可能减小噪声和个体差异的干扰，对 AB 列的误差进行有效控制，对正常 GC 含量的数据标为 0，其余数据（偏差较大）标为

1, 表示异常 GC 值对模型造成的影响。

3.特征标准化

在建立特征方程前需要将不同特征进行 Z-score 标准化处理以消除量纲的影响, 标准化公式如下:

$$Z = \frac{x-u}{\sigma} \quad (22)$$

4.特征工程的建立

本研究接下来要考虑 X 染色体、21 号、18 号、13 号染色体的 Z 值、GC 含量、读数段及相关比例、BMI 等因素, 所以选取附件中 K 到 AA 列 (因为女胎没有 Y 染色体, 所以 U、V 列空缺) 共 15 组特征量, 建立特征工程。数据集以 70-30 的比例划分为训练集与测试集, 其中利用 70% 的数据用于训练, 30% 的数据用于测试。

5.4.2 逻辑回归模型的建立与求解

1.模型训练参数

最大迭代次数 epochs=1000

2.模型的建立

将以上所有提到的 15 组特征量合并在一个 X 向量中, 假设这些数据特征与分类结果女胎染色体非整倍体类型 Y (取值为 0、1、2) 之间存在一定关系, 分别预测 Y=0,1,2 的概率。将线性回归方程带入 Sigmoid 函数, 从而得到逻辑回归的预测函数如下:

$$P(Y=0,1,2|X) = \sigma(w^T X + b) = \frac{1}{1 + e^{-(w^T X + b)}} \quad (23)$$

3.决策边界

设定一个阈值, 这里选择通用值 0.5 为最终分类做边界。

4.模型的求解

通过以上建立的逻辑回归模型, 将给出的女胎相关信息再次带入模型, 得到预测结果。

5.模型的评估

分析模型评估结果: 准确率、召回率和 F1 分数, 是评估模型性能的重要方式
准确率:

$$P = \frac{TP}{TP+FP} \quad (24)$$

召回率:

$$R = \frac{TP}{TP+FN} \quad (25)$$

F₁ 分数

$$F_1 = \frac{2PR}{P+R} \quad (26)$$

在 0.5 的阈值条件下, $P=0.8889$, $R=0.0889$, $F_1=0.8506$ 。

由以上三值可以得到逻辑回归模型准确率较高, 模型在整体数据上表现良好; 召回率也较高, 说明模型可以很好地识别正类样本; 且 F1 分数可以说明模型在准确率和召

回率之间取得了一定的平衡。但是又考虑到 Logistic 回归对噪声较为敏感，而我们的特征数据中的 C-G 含量数据存在部分异常含量数据，会对回归造成较大影响。

因此我们选择继续使用一种能够更好处理特征关系，具有强噪声耐受性，易解释特征重要性的随机森林机器学习方法。

5.4.3 随机森林模型的建立与求解

1.模型训练的参数

决策数的数量 $N=200$

深度 $depth=null$ （没有设定决策树深度，由机器学习自主确定）

内部节点再划分所需要的最小样本数：2

叶子节点的最小样本数：1

2.模型的建立

对于上述所提到的 15 个特征，训练集 70% 的数据，构建 N 棵决策树，每棵树都会给出对应的预测女胎染色体非整倍体类型，森林的最终预测结果是所有树预测结果的众数（即得票最多的那个类别）。

$$\text{Final Prediction} = \text{Mode}(Tree_1(x), Tree_2(x), \dots, Tree_T(x)) \quad (27)$$

3.决策边界：

设定一个阈值，这里同样选择通用值 0.5 为最终分类做边界。

4.模型的求解

这里求得预测数据如下：

表 7 随机森林预测结果

序号	指标	13 号染色体的 Z 值	18 号染色体的 Z 值	21 号染色体的 Z 值	...	重新分类	最高概率对应类别
1		0.8933	-0.0942	0.7756	...	正常	正常
2		1.3551	0.8937	-1.3059	...	正常	正常

5.模型的评估

在 0.5 的阈值条件下，准确率=0.9125，召回率=0.9125，F1 分数=0.8706。

由以上三值可以得到随机森林模型相较逻辑回归模型具有更高的准确率，召回率以及 F1 分数，说明随机森林模型相较逻辑回归模型在此时具有更好的性能，能更好的对女胎染色体非整倍体类型进行分类。因此要对已知女胎特征时，可以将特征数据带入建立的随机森林模型进行分类预测，进而判定女胎是否异常。

六、模型的评价、改进与推广

6.1 模型的优点

1.在相关性分析阶段，选择非线性混合效应模型可以很好的应用于本题一位孕妇多次测量的层次性较强的数据，因而模型与本文拟合度较高。

2.模型参考大量相关领域医学文献，引入实验室测量数据来衡量误差大小，相较随机模拟误差生成的数据更为可靠，有利于对检测误差进行更准确的分析。

3.引入逻辑回归，通过与 Y 染色体浓度强相关特征的拟合，对其进行预测修正，可以有效降低真实值与实验值之间的误差。

6.2 模型的改进

1.风险最小化模型的建立较为单一简洁，虽一定程度上简化了运算，但模型的普适性稳健性不高，可以引入普适性更强的 Cox 比例风险模型，对于医学领域的生存分析具有更高的灵活性和稳健性。

2.采用优化模型进行 BMI 分组相较聚类分析在本题可以有效减少 BMI 相较过大的数据聚类在一起，但是单独进行优化约束，约束条件不具备应有的约束力，会影响对目标函数的精准优化，因此可以考虑联立分组优化和风险优化，增强约束有效性。

3.相较直接采用机器学习算法进行分类，还可以使用为特定医学数据集设计的专用分类模型对本文女胎异常判定进行分析。

6.3 模型的推广

本文所建立的风险最小化模型用于求解孕周最佳检测时点，改模型可以推广运用到其他医学、生物学检测领域的相关问题中，也可以作为其他风险优化问题的起点，为解决这些问题提供思路。

七、参考文献

- [1] Benn P, Zhang J, Lyons D, Xu W, Leonard S, Demko Z. Accuracy of fetal fraction measurements in a single-nucleotide polymorphism-based noninvasive prenatal test. *Prenat Diagn.* 2024; 44(10): 1218-1224. <https://doi.org/10.1002/pd.6634>
- [2] 张淑棋. NIPT 在双胎妊娠染色体非整倍体筛查效能的临床研究[D]. 福建医科大学, 2023.
- [3] 韦宝华. 基于自适应弹性网的纵向数据混合效应模型及其应用[D]. 广西师范大学, 2025.
- [4] 夏丹玲,刘娟. 基于 Logistic 回归与随机森林模型的老年脑梗死患者跌倒恐惧危险因素分析 [J]. 东南大学学报(医学版), 2025, 44 (04): 654-660.
- [5] 廖高明,邓一荣,刘丽丽,等. 基于蒙特卡洛模拟的典型电镀场地氯代烃污染特征与风险评价 [J]. 环境科学研究, 2025, 38 (08): 1814-1825.
- [6] DeepSeek, DeepSeek-R1-0528, 深度求索 (DeepSeek), 2025-09-07

附录

附录 1

介绍：支撑材料的文件列表

1. 第一问相关系数.py
2. 第一问散点图.py
3. 第一问多项式回归分析.py
4. 第一问混合效应模型.py
5. 第二问聚类 km.py
6. 第二问风险量化.py
7. 第二问误差分析.py
8. 第三问优化定区间.py
9. 第三问误差分析.py
10. 第三问风险量化.py
11. 第四问逻辑回归.py
12. 第四问随机森林.py
13. 第四问 XGBoost.py
14. 第一问.xlsx
15. 问题一混合效应模型数据.xlsx
16. 第二问风险量化.xlsx
17. 第二问聚类分析.xlsx
18. 第三问聚类数据.xlsx
19. 第三问风险量化.xlsx
20. 预测结果.xlsx
21. 全部样本预测结果_xgboost.xlsx
22. 全部样本预测结果.xlsx
23. 蒙特卡洛模拟详细结果.xlsx
24. 第四问数据.xlsx
25. 测试集预测结果.xlsx
26. BMI 自动分组结果.xlsx
27. BMI 分组分析结果.xlsx
28. bmi_auto_groups.png
29. bmi_group_features.png
30. clustering_evaluation.png
31. silhouette_scores.png
32. Y 染色体浓度散点图.png
33. Y 染色体浓度显著变量散点图.png
34. Y 染色体浓度相关系数条形图.png
35. Y 染色体浓度箱线图.png
36. Y 染色体浓度箱线图_处理前后对比.png
37. 残差分布.png
38. 第三问 BMI 分区.png
39. 多项式回归残差图.png

- 40. 多项式回归拟合曲线.png
- 41. 多项式回归预测对比图.png
- 42. 风险期望分布.png
- 43. 混合效应模型预测的对比图.png
- 44. 蒙特卡洛模拟结果比较.png
- 45. 斯皮尔曼相关系数热力图.png
- 46. 随机截距效应.png
- 47. 随机森林回归残差图.png
- 48. 随机森林回归预测对比图.png
- 49. 随机森林特征重要性.png
- 50. 随机森林学习曲线.png
- 51. AI 使用详情

附录 2

介绍：相关系数（Python）

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

plt.rcParams['font.sans-serif'] = ['SimHei'] # 中文设置
plt.rcParams['axes.unicode_minus'] = False

file_path = "第一问.xlsx"
df = pd.read_excel(file_path)

if '怀孕次数' in df.columns:
    df['怀孕次数'] = df['怀孕次数'].replace('≥3', 3).astype(float) #对怀孕次数数据进行数据预处理

#散点图建立
fig, axes = plt.subplots(1, 3, figsize=(15, 5))
fig.suptitle('Y 染色体浓度与显著相关因素的关系散点图', fontsize=16)

if all(col in df.columns for col in ['检测孕周', 'Y 染色体浓度']):
    axes[0].scatter(df['检测孕周'], df['Y 染色体浓度'], alpha=0.5)
    axes[0].set_xlabel('检测孕周')
    axes[0].set_ylabel('Y 染色体浓度')
    axes[0].set_title('Y 染色体浓度 vs 检测孕周')

if all(col in df.columns for col in ['孕妇 BMI', 'Y 染色体浓度']):
    axes[1].scatter(df['孕妇 BMI'], df['Y 染色体浓度'], alpha=0.5)
```



```

axes[1].set_xlabel('孕妇 BMI')
axes[1].set_ylabel('Y 染色体浓度')
axes[1].set_title('Y 染色体浓度 vs 孕妇 BMI')

if all(col in df.columns for col in ['年龄', 'Y 染色体浓度']):
    axes[2].scatter(df['年龄'], df['Y 染色体浓度'], alpha=0.5)
    axes[2].set_xlabel('孕妇年龄')
    axes[2].set_ylabel('Y 染色体浓度')
    axes[2].set_title('Y 染色体浓度 vs 孕妇年龄')

plt.tight_layout()
plt.savefig('Y 染色体浓度显著变量散点图.png', dpi=300, bbox_inches='tight')
plt.show()

# 斯皮尔曼相关系数的计算
if 'Y 染色体浓度' in df.columns:
    spearman = df.corr(method='spearman')
    print(spearman['Y 染色体浓度'].sort_values(ascending=False))

# 热力图
plt.figure(figsize=(12, 10))
heatmap = sns.heatmap(spearman,
                       annot=True,
                       cmap='RdBu',
                       center=0,
                       square=True,
                       fmt='.3f',
                       cbar_kws={"shrink": .8},
                       annot_kws={"size": 15},
                       linewidths=0.5)

plt.title('斯皮尔曼相关系数热力图', fontsize=16, pad=20)
plt.xticks(rotation=45, ha='right', fontsize=12)
plt.yticks(rotation=0, fontsize=12)
plt.tight_layout()
plt.savefig('斯皮尔曼相关系数热力图.png', dpi=300, bbox_inches='tight')
plt.show()

```

附录 3

介绍：多项式回归模型（Python）

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import statsmodels.api as sm

```

```

import re

from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import cross_val_score, train_test_split
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.pipeline import Pipeline
from statsmodels.stats.outliers_influence import variance_inflation_factor

plt.rcParams['font.sans-serif'] = ['SimHei'] # 中文设置
plt.rcParams['axes.unicode_minus'] = False

file = "问题一.xlsx"
df = pd.read_excel(file)

if '怀孕次数' in df.columns:
    df['怀孕次数'] = df['怀孕次数'].replace('≥3', 3).astype(float) #对怀孕次数数据进行数据预处理

# 多項式回歸分析
lie = ['Y 染色体浓度', '检测孕周', '孕妇 BMI', '年龄']
misslie = [col for col in lie if col not in df.columns]

if misslie:
    print(f'无法建立多项式回归模型：缺少以下列 {misslie}')
else:
    X = df[['检测孕周', '孕妇 BMI', '年龄']]
    y = df['Y 染色体浓度']

    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42) #數據集劃分

    nums = [1, 2, 3] #分析1~3次多項式的情況
    results = {}
    for degree in nums:
        model = Pipeline([
            ('poly', PolynomialFeatures(degree=degree)),
            ('scaler', StandardScaler()),
            ('linear', LinearRegression())
        ])

        model.fit(X_train, y_train)
        y_pred = model.predict(X_test)

        mse = mean_squared_error(y_test, y_pred)
        r2 = r2_score(y_test, y_pred)
        cv_scores = cross_val_score(model, X, y, cv=3, scoring='r2') #交叉驗證

```

```

results[degree] = {
    'model': model,
    'mse': mse,
    'r2': r2,
    'cv_mean': np.mean(cv_scores),
    'cv_std': np.std(cv_scores)
}

print(f'MSE: {mse:.6f}')
print(f'R2: {r2:.4f}')
print(f'交叉验证 R2: {np.mean(cv_scores):.4f} ( $\pm$ {np.std(cv_scores):.4f})')

if degree == 1:
    feature_names = X.columns.tolist()
    coef = model.named_steps['linear'].coef_
    intercept = model.named_steps['linear'].intercept_

    print("\n 回归系数:")
    print(f'截距: {intercept:.6f}')
    for i, name in enumerate(feature_names):
        print(f'{name}: {coef[i + 1]:.6f}')

# 最後選擇三次多項式
fig, axes = plt.subplots(1, 3, figsize=(15, 5))
fig.suptitle('各变量与 Y 染色体浓度的关系及三次多项式拟合', fontsize=16)

variables = ['检测孕周', '孕妇 BMI', '年龄']
for i, var in enumerate(variables):
    ax = axes[i]
    ax.scatter(df[var], y, alpha=0.5, label='实际值')
    x_range = np.linspace(df[var].min(), df[var].max(), 100).reshape(-1, 1)
    poly_model = Pipeline([
        ('poly', PolynomialFeatures(degree=3)),
        ('scaler', StandardScaler()),
        ('linear', LinearRegression())
    ])

    poly_model.fit(df[[var]], y)
    y_poly = poly_model.predict(x_range)
    ax.plot(x_range, y_poly, color='red', label='三次多项式拟合')
    ax.set_xlabel(var)
    ax.set_ylabel('Y 染色体浓度')
    ax.set_title(f'Y 染色体浓度 vs {var}')
    ax.legend()

plt.tight_layout()
plt.savefig('三次多项式回归拟合曲线.png', dpi=300, bbox_inches='tight')
plt.show()

```

```

poly = PolynomialFeatures(degree=3)
X_poly = poly.fit_transform(X)
linr = LinearRegression()
linr.fit(X_poly, y)
feature_names = poly.get_feature_names_out(['检测孕周', '孕妇 BMI', '年龄'])

equation = f"Y 染色体浓度 = {linr.intercept_:.6f}"

for i, coef in enumerate(linr.coef_):
    if i == 0:
        continue
    if abs(coef) > 1e-6:
        feature_name = feature_names[i]
        feature_name = re.sub(r'\^(d+)', r'^\1', feature_name)
        feature_name = feature_name.replace(' ', '.')

        if coef > 0:
            equation += f" + {coef:.6f} · {feature_name}"
        else:
            equation += f" - {abs(coef):.6f} · {feature_name}"
print("三次多项式回归方程:")
print(equation)

# F 检验和 t 检验
X_poly_with_const = sm.add_constant(X_poly)

# 创建 OLS 模型
ols_model = sm.OLS(y, X_poly_with_const)
ols_results = ols_model.fit()
print(ols_results.summary())

# F 检验
f_value = ols_results.fvalue
f_pvalue = ols_results.f_pvalue
print(f"\nF 检验结果:")
print(f"F 统计量: {f_value:.4f}")
print(f"P 值: {f_pvalue:.4e}")

# t 检验
print(f"\nT 检验结果 (部分显著变量):")
pvalues = ols_results.pvalues
tvalues = ols_results.tvalues

significant_vars = pvalues[pvalues < 0.05].index
print("显著变量 (p < 0.05):")
for var in significant_vars:
    if var != 'const':

```

```
print(f'{var}: t 值 = {tvalues[var]:.4f}, p 值 = {pvalues[var]:.4e}')
```

附录 4

介绍：混合效应模型（Python）

```
import matplotlib.pyplot as plt
import pandas as pd
import statsmodels.api as sm
import numpy as np
from statsmodels.regression.mixed_linear_model import MixedLM
from sklearn.metrics import mean_squared_error, r2_score
from scipy import stats

# 为代码设置中文字体，以便于作图
plt.rcParams['font.sans-serif'] = ['SimHei'] #设置 SimHei 的字体样式
plt.rcParams['axes.unicode_minus'] = False

file = "问题一混合效应模型数据.xlsx" #从整理后的 excel 里读取数据
df = pd.read_excel(file)

# 进行非线性混合效应模型的建立
required_names = ['Y 染色体浓度', '检测孕周', '孕妇 BMI', '年龄', '孕妇代码']
missing_names = [col for col in required_names if col not in df.columns]

if missing_names:
    print(f'由于缺少这列数据而不能运行 {missing_names}')
else:
    X = df[['检测孕周', '孕妇 BMI', '年龄']]
    y = df['Y 染色体浓度']
    groups = df['孕妇代码'] # 对孕妇进行分组

    # 截距项的添加
    X = sm.add_constant(X)

    ols_model = sm.OLS(y, X)
    ols_result = ols_model.fit()
    print(ols_result.summary()) #构建一个普通的模型进行对比

    # 使用最大似然估计拟合模型
    # 这里我们使用随机截距模型，即每个孕妇有自己的截距
    model = MixedLM(y, X, groups=groups)
```

```

result = model.fit(method='powell') # 使用 Powell 方法优化, 更稳定

# 打印 OLS Regression 模型的结果
print(result.summary())

# 对固定效应的显著性检验结果进行提取并格式化
fixed_effects = pd.DataFrame({
    '系数': result.fe_params,
    '标准误': result.bse_fe,
    'z 值': result.tvalues,
    'P 值': result.pvalues
})
print(fixed_effects)

# 使用似然比检验去进行随机效应的显著性检验
lr_statistic = 2 * (result.llf - ols_result.llf)
p_value = stats.chi2.sf(lr_statistic, 1)

print(f'混合模型的对数似然值: {result.llf:.4f}')
print(f'线性模型的对数似然值: {ols_result.llf:.4f}')
print(f'似然比的统计量: {lr_statistic:.4f}')
print(f'P 值: {p_value:.4f}')

# 获取预测值
y_pred = result.fittedvalues

# 对混合效应模型进行评估
mse = mean_squared_error(y, y_pred)
r2 = r2_score(y, y_pred)

print(f'\n 模型评估:')
print(f'均方误差(MSE): {mse:.6f}')
print(f'决定系数(R²): {r2:.4f}')

fixed_intercept = result.fe_params['const']
fixed_slopes = result.fe_params.drop('const')

print(f'固定效应的截距: {fixed_intercept:.6f}')
print("固定效应的斜率:")
for var, coef in fixed_slopes.items():
    print(f'    {var}: {coef:.6f}')

random_effects = result.random_effects

group_to_re = {}
for group, effect in random_effects.items():
    if hasattr(effect, 'iloc'):

```

```

        effect_value = effect.iloc[0] if len(effect) > 0 else 0
    elif hasattr(effect, '__len__') and not isinstance(effect, str):
        effect_value = effect[0] if len(effect) > 0 else 0
    else:
        effect_value = effect
    group_to_re[group] = effect_value

# 为每个观测值添加随机的效应
df['随机效应的截距'] = df['孕妇代码'].map(group_to_re)
df['总截距'] = fixed_intercept + df['随机效应的截距']

# 4. 计算每个观测值的固定效应部分
df['固定效应部分'] = fixed_intercept
for var in fixed_slopes.index:
    df['固定效应部分'] += fixed_slopes[var] * df[var]

df['模型预测值'] = df['固定效应部分'] + df['随机效应的截距']
df['残差'] = df['Y 染色体浓度'] - df['模型预测值']

# 对各个组成部分的统计量进行计算
print(f"\n 各个组成部分的统计量:")
print(f'固定效应的部分均值: {df["固定效应部分"].mean():.6f}')
print(f'随机效应的截距均值: {df["随机效应的截距"].mean():.6f}')
print(f'随机效应的截距标准差: {df["随机效应的截距"].std():.6f}')
print(f'残差的均值: {df["残差"].mean():.6f}')
print(f'残差的标准差: {df["残差"].std():.6f}')

# 对随机效应截距的分布图以及预测值与实际值对比图进行绘制
plt.figure(figsize=(10, 5))
plt.hist(df['随机效应的截距'].dropna(), bins=30, alpha=0.7)
plt.axvline(x=0, color='r', linestyle='--', label='均值 (0)')
plt.xlabel('随机效应的截距')
plt.ylabel('频数')
plt.title('随机效应截距的分布')
plt.legend()
plt.savefig('随机效应截距的分布.png', dpi=350, bbox_inches='tight')
plt.show()

plt.figure(figsize=(10, 5))
plt.scatter(y, y_pred, alpha=0.3)
plt.plot([y.min(), y.max()], [y.min(), y.max()], 'r--', lw=2)
plt.xlabel('实际值')
plt.ylabel('预测值')
plt.title('混合效应模型的预测值与实际值对比')
plt.savefig('混合效应模型预测的对比图.png', dpi=350, bbox_inches='tight')

```

```
plt.show()
```

附录 5

介绍：第二问风险量化（Python）

```
import pandas as pd
import numpy as np
from scipy.optimize import minimize

# 根据上述所得到的BMI 分界点进行划分区域
bmis = [20.703125, 31.21763, 33.197865, np.inf]
bmi_nums = [1, 2, 3]

# 根据题目已给附件区计算在不同BMI 区间内的y 染色体浓度 $\geq 4\%$ 概率
probabilities = {
    1: 235 / 269,
    2: 472 / 525,
    3: 221 / 276
}

# 读取数据
file = "第二问风险量化.xlsx"
data = pd.read_excel(file)
# 为excel 表内特征进行命名
bmi_names = '孕妇 BMI/BMIg'
gestational_names = '检测孕周/R(t)'
bmi_data = data[bmi_names]
gestations_weeks = data[gestational_names]

data['BMI_group'] = pd.cut(bmi_data, bins=bmis, labels=bmi_nums, right=False)

ti = 1 # 同一位孕妇每次检测的间隔时间
maxweek = 25 # 题目给出 10-25 周的最大孕周
checksmax = 5 # 用已知数据“抽血次数”确立最大的检查次数

# 对递归的风险期望函数  $E(R(t)|t, BMIg)$  进行定义
def minrisk(t, p, ti=1, maxweek=25, check_count=1, checksmax=5):
    if check_count > checksmax:
        return t

    # 当  $t + \Delta t >$  最大孕周时，那么只考虑当前时间点的风险
    if t + ti > maxweek:
        return t

    # 递归计算，并且增加检查次数
```



```

    next_risk = minrisk(t + ti, p, ti, maxweek, check_count + 1, checksmax)

    # 返回目前所在时间点的风险期望
    return t + (1 - p) * next_risk

def expectrisk(t, group_data, ti=1, maxweek=25, checksmax=5):
    total_risk = 0
    Ng = len(group_data)

    if Ng == 0:
        return np.inf

    # 获取每个不同的BMI组的概率
    group = group_data['BMI_group'].iloc[0]
    p = probabilities[group]

    # 计算最后的递归的不同时间点的风险期望
    risk = minrisk(t, p, ti, maxweek, 1, checksmax)

    return risk

# 计算每个不同的BMI区间的最小风险期望及其对应的孕周
results = {}
for group in bmi_nums:
    group_data = data[data['BMI_group'] == group]

    if len(group_data) == 0:
        print(f'BMI 区间 {group} 没有数据')
        continue

    group_min_week = group_data[gestational_names].min()
    group_maxweek = group_data[gestational_names].max()
    print(f'BMI 区间 {group} 的孕周范围： {group_min_week:.2f} - {group_maxweek:.2f} 周')
    initial_guess = group_data[gestational_names].median()

    # 优化边界的设立
    bounds = [(group_min_week, group_maxweek)]

    # 最后的最小化风险期望函数
    result = minimize(
        lambda t: expectrisk(t[0], group_data),
        x0=[initial_guess],
        bounds=bounds,
        method='L-BFGS-B'
    )

# 结果答案

```

```

min_risk = result.fun
best_t = result.x[0]

# 计算该组所有样本在最佳t处的风险期望
risks = []
for _, row in group_data.iterrows():
    t_val = row[gestational_names]
    p = probabilities[group]
    risk_val = minrisk(t_val, p, ti, maxweek, 1, checksmax)
    risks.append(risk_val)

mean_risk = np.mean(risks)
std_risk = np.std(risks)

results[group] = {
    'best_t': best_t,
    'min_risk': min_risk,
    'mean_risk': mean_risk,
    'std_risk': std_risk,
    'num_samples': len(group_data),
    'min_week': group_min_week,
    'maxweek': group_maxweek
}

print("\n 各区间的风险期望分析结果:")
for group, result in results.items():
    print(f"\nBMI 区间 {group}:")
    print(f"  区间样本的数量: {result['num_samples']}")
    print(f"  孕周的范围: {result['min_week']:.2f} - {result['maxweek']:.2f} 周")
    print(f"  最佳检测的孕周: {result['best_t']:.4f}")
    print(f"  最小风险的期望: {result['min_risk']:.4f}")
    print(f"  平均风险的期望: {result['mean_risk']:.4f}")
    print(f"  风险期望的标准差: {result['std_risk']:.4f}")

```

附录 7

介绍：第三问风险量化（Python）

```

import pandas as pd
import numpy as np
from scipy.optimize import minimize
from sklearn.linear_model import LogisticRegression
import warnings

```

```

warnings.filterwarnings('ignore')

# 读取 Excel 数据
file = "第三问风险量化.xlsx"
data = pd.read_excel(file)

# 打印列名以确认实际列名
print("Excel 文件的列名:")
print(data.columns.tolist())

# 根据实际列名调整以下变量
# 请根据打印出的列名修改以下变量
bmi_column = '孕妇 BMI/BMIg' # 请根据实际列名修改
gestational_weeks_column = '检测孕周/R(t)' # 请根据实际列名修改
y_chromosome_column = 'Y 染色体浓度' # 请根据实际列名修改
age_column = '年龄' # 请根据实际列名修改
height_column = '身高' # 请根据实际列名修改
weight_column = '体重' # 请根据实际列名修改

# 数据清洗：处理异常值和缺失值
# 移除检测孕周为"-"的行
data = data[data[gestational_weeks_column] != '-']
# 将检测孕周转换为数值类型
data[gestational_weeks_column] = pd.to_numeric(data[gestational_weeks_column],
errors='coerce')
# 移除转换失败的行
data = data.dropna(subset=[gestational_weeks_column])

# 定义 BMI 区间
bmis = [20.703125, 30.18801475, 31.8608855, 33.9036355, np.inf]
bmi_nums = [1, 2, 3, 4]

# 为数据添加 BMI 分组
data['BMI_group'] = pd.cut(data[bmi_column], bins=bmis, labels=bmi_nums,
right=False)

# 获取整个数据集的孕周范围
min_week = data[gestational_weeks_column].min()
max_week = data[gestational_weeks_column].max()
print(f"\n 整个数据集的孕周范围: {min_week:.2f} - {max_week:.2f} 周")

# 准备逻辑回归的特征和目标变量
feature_columns = [bmi_column, age_column, height_column, weight_column]

# 创建二值目标变量：Y 染色体浓度是否>=4%
data['Y_binary'] = (data[y_chromosome_column] >= 0.04).astype(int)

```

```

# 准备逻辑回归数据
X = data[feature_columns]
y = data['Y_binary']

# 训练逻辑回归模型
logreg = LogisticRegression(max_iter=1000)
logreg.fit(X, y)

# 获取回归系数
beta0 = logreg.intercept_[0]
beta_coefs = logreg.coef_[0]

# 打印逻辑回归系数
print("\n 逻辑回归系数:")
print(f'截距项  $\beta_0$ : {beta0:.6f}')
for i, feature in enumerate(feature_columns):
    print(f'特征 '{feature}' 的系数  $\beta_{i+1}$ : {beta_coefs[i]:.6f}')

# 定义概率函数  $p(Y=1|t, X)$ 
def probability_t(t, X_features):
    # X_features 应该是一个包含 BMI, age, height, weight 的数组
    linear_combination = beta0 + beta_coefs[0] * t + np.dot(beta_coefs[1:], X_features)
    return 1 / (1 + np.exp(-linear_combination))

# 定义递归的风险期望函数  $E(R(t)|t, BMIg)$ 
def recursive_expected_risk(t, X_features, delta_t=2, max_week=25, check_count=1, max_checks=5):
    # 如果检查次数超过限制, 则只考虑当前时间点的风险
    if check_count > max_checks:
        return t

    # 如果  $t + \Delta t > 25$ , 则只考虑当前时间点的风险
    if t + delta_t > max_week:
        return t

    # 计算当前时间点的成功概率
    p = probability_t(t, X_features)

    # 递归计算下一时间点的风险期望, 增加检查次数
    next_risk = recursive_expected_risk(t + delta_t, X_features, delta_t, max_week,
                                         check_count + 1, max_checks)

    # 返回当前时间点的风险期望
    return t + (1 - p) * next_risk

```

```

# 定义风险期望函数  $E(R(t)|t, BMI_g)$ 
def expected_risk(t, group_data, delta_t=2, max_week=25, max_checks=5):
    total_risk = 0
    Ng = len(group_data)

    if Ng == 0:
        return np.inf

    for _, row in group_data.iterrows():
        # 提取特征: 年龄, 身高, 体重 (BMI 已经在分组中考虑了)
        X_features = row[[age_column, height_column, weight_column]].values

        # 计算递归的风险期望
        risk_i = recursive_expected_risk(t, X_features, delta_t, max_week, 1,
max_checks)
        total_risk += risk_i

    # 返回平均风险期望
    return total_risk / Ng

# 对于每个 BMI 分组, 找到最小化风险期望的  $t$ 
results = {}
for group in bmi_nums:
    group_data = data[data['BMI_group'] == group]

    if len(group_data) == 0:
        print(f'BMI 区间 {group} 没有数据')
        continue

    # 获取该分组的孕周范围
    group_min_week = group_data[gestational_weeks_column].min()
    group_max_week = group_data[gestational_weeks_column].max()
    print(f'BMI 区间 {group} 的孕周范围: {group_min_week:.2f} -
{group_max_week:.2f} 周')

    # 使用分组数据中的中位数孕周作为初始猜测
    initial_guess = group_data[gestational_weeks_column].median()

    # 使用该分组的孕周范围作为优化边界
    bounds = [(group_min_week, group_max_week)]

    # 最小化风险期望函数
    result = minimize(
        lambda t: expected_risk(t[0], group_data),
        x0=[initial_guess],

```

```

        bounds=bounds,
        method='L-BFGS-B'
    )

    # 存储结果
    min_risk = result.fun
    best_t = result.x[0]

    results[group] = {
        'best_t': best_t,
        'min_risk': min_risk,
        'num_samples': len(group_data),
        'min_week': group_min_week,
        'max_week': group_max_week
    }

# 打印结果
print("\n 各区间的风险期望最小化结果:")
for group, result in results.items():
    print(f"\nBMI 区间 {group}:")
    print(f"  区间样本数量: {result['num_samples']}")
    print(f"  孕周范围: {result['min_week']:.2f} - {result['max_week']:.2f} 周")
    print(f"  最佳检测孕周: {result['best_t']:.4f}")
    print(f"  最小风险期望: {result['min_risk']:.4f}")

```

附录 8

介绍：第三问误差分析（Python）

```

import pandas as pd
import numpy as np
from sklearn.linear_model import LogisticRegression
from scipy.optimize import minimize
import warnings

warnings.filterwarnings('ignore')

# 对第三问整理数据进行读取
file = "第三问风险量化.xlsx"
data = pd.read_excel(file)

# 为所有特征量进行重新命名操作，方便后续代码运算
bmizhi = '孕妇 BMI/BMIg'
jiance_weeks = '检测孕周/R(t)'

```

```

y_chromosome_zhi = 'Y 染色体浓度'
age_zhi = '年龄'
height_zhi = '身高'
weight_zhi = '体重'

#数据预处理
data = data[data[jiance_weeks] != '-']
data[jiance_weeks] = pd.to_numeric(data[jiance_weeks], errors='coerce')
data = data.dropna(subset=[jiance_weeks])

# 根据前面分组情况写出每组边界
bmis = [20.703125, 30.18801475, 31.8608855, 33.9036355, np.inf]
bmi_nums = [1, 2, 3, 4]

# 为每组 BMI 值进行界限
data['BMI_group'] = pd.cut(data[bmizhi], bins=bmis, labels=bmi_nums, right=False)

# 获得整个孕周的范围值
min_week = data[jiance_weeks].min()
max_week = data[jiance_weeks].max()
print(f"\n 所有数据的孕周范围: {min_week:.2f} - {max_week:.2f} 周")

# 对逻辑回归的值进行预准备
feature_zhi = [bmizhi, age_zhi, height_zhi, weight_zhi]

# 定义一个蒙特卡洛模拟的模型, 用于模拟误差
def monte_carlo_sim(data0, simulations_num=100, error_ranges=0.0126):
    all_results = []
    for sim in range(simulations_num):
        print(f"\n 正在进行第 {sim + 1}/{simulations_num} 次模拟...")
        sim_data = data0.copy()
        error_factors = np.random.uniform(-error_ranges, error_ranges,
size=len(sim_data))
        sim_data[y_chromosome_zhi] = sim_data[y_chromosome_zhi] * (1 +
error_factors)
        # 保证染色体浓度在 0-1 之间
        sim_data[y_chromosome_zhi] = sim_data[y_chromosome_zhi].clip(0, 1)
        sim_data['Y_binary'] = (sim_data[y_chromosome_zhi] >= 0.04).astype(int)

        # 逻辑回归数值准备
        X = sim_data[feature_zhi]
        y = sim_data['Y_binary']

        # 对逻辑回归模型进行训练
        try:
            logreg = LogisticRegression(max_iter=1000)
            logreg.fit(X, y)

```

```

beta0 = logreg.intercept_[0]
beta_coefs = logreg.coef_[0]

# 定义概率函数  $p(Y=1|t,X)$ 
def probability_t(t, X_features):
    linear_combination = beta0 + beta_coefs[0] * t +
np.dot(beta_coefs[1:], X_features)
    return 1 / (1 + np.exp(-linear_combination))

# 定义递归的风险期望函数  $E(R(t)|t,BMIg)$ 
def recursive_expected_risk(t, X_features, delta_t=2, max_week=25,
check_count=1, max_checks=5):
    if check_count > max_checks:
        return t

    if t + delta_t > max_week:
        return t

    p = probability_t(t, X_features)
    next_risk = recursive_expected_risk(t + delta_t, X_features, delta_t,
max_week, check_count + 1,
max_checks)

    return t + (1 - p) * next_risk

# 定义风险期望函数  $E(R(t)|t,BMIg)$ 
def expected_risk(t, group_data, delta_t=2, max_week=25,
max_checks=5):
    total_risk = 0
    Ng = len(group_data)

    if Ng == 0:
        return np.inf

    for _, row in group_data.iterrows():
        X_features = row[['age_zhi', 'height_zhi', 'weight_zhi']].values
        risk_i = recursive_expected_risk(t, X_features, delta_t,
max_week, 1, max_checks)
        total_risk += risk_i

    return total_risk / Ng

# 对于每个 BMI 分组，找到最小化风险期望的  $t$ 
sim_results = {}
for group in bmi_nums:
    group_data = sim_data[sim_data['BMI_group'] == group]

    if len(group_data) == 0:

```



```

        print(f'BMI 区间 {group} 没有数据")
        continue

    mingroup_week = group_data[jiance_weeks].min()
    maxgroup_week = group_data[jiance_weeks].max()

    # 使设置一个初始值进行预测
    initial_guess = group_data[jiance_weeks].median()

    # 将保密分界也作为一个边界限制条件
    bounds = [(mingroup_week, maxgroup_week)]

    # 要使风险最小的目标函数
    result = minimize(
        lambda t: expected_risk(t[0], group_data),
        x0=[initial_guess],
        bounds=bounds,
        method='L-BFGS-B'
    )

    # 将结果存起来
    min_risk = result.fun
    best_t = result.x[0]

    sim_results[group] = {
        'best_t': best_t,
        'min_risk': min_risk,
        'num_samples': len(group_data)
    }

    all_results.append(sim_results)

except Exception as e:
    print(f'第 {sim + 1} 次模拟中出现错误: {e}")
    continue

return all_results

# 执行100次蒙特卡洛模拟
simulations_num = 100
mc_results = monte_carlo_sim(data, simulations_num)

# 对蒙特卡洛模拟的结果进行分析
avg_results = {}
for group in bmi_nums:
    best_times = []
    min_risks = []

```

```

for result in mc_results:
    if group in result:
        best_times.append(result[group]['best_t'])
        min_risks.append(result[group]['min_risk'])

if best_times:
    avg_best_t = np.mean(best_times)
    std_best_t = np.std(best_times)
    avg_min_risk = np.mean(min_risks)

    avg_results[group] = {
        'avg_best_t': avg_best_t,
        'std_best_t': std_best_t,
        'avg_min_risk': avg_min_risk,
        'simulations_num': len(best_times)
    }

# 打印蒙特卡洛模拟的结果
print("\n 蒙特卡洛模拟的结果 (考虑 Y 染色体浓度±1.26%误差):")
for group, result in avg_results.items():
    print(f"\nBMI 区间 {group}:")
    print(f"    模拟次数: {result['simulations_num']}")
    print(f"    平均的最佳检测孕周 : {result['avg_best_t']:.4f} ± {result['std_best_t']:.4f}")
    print(f"    平均的最小风险期望: {result['avg_min_risk']:.4f}")

```